

PYTHON PRO MODE

A Ponte Definitiva: Como Deixar os Blocos do Scratch e Dominar o Código
Real das Big Techs!

>_ **Nível Recomendado: 13 a 17 Anos | Dev Mode Unlocked**

Módulo 1: O Nível Secreto dos Desenvolvedores

Por que a transição dos blocos para o texto é o seu maior superpoder na tecnologia!



Você Já Sabe Programar!

⚡ +50 XP

- ⚡ No Scratch, você aprendeu a lógica: Loops, Variáveis, Condições e Eventos.
- ⚡ Python usa exatamente a mesma lógica, mas sem os encaixes coloridos.
- ⚡ Você não está começando do zero; você está apenas trocando a interface gráfica pelo terminal profissional!

Fact Check: A lógica de um `se/então` no Scratch é a mesma que controla o algoritmo do TikTok, YouTube e OpenAI!





Por Que Python É a Linguagem #1?

⚡ +50 XP



Inteligência Artificial

É a linguagem principal usada para criar o ChatGPT, robôs autônomos e visão computacional.



Desenvolvimento de Games

Usado no Pygame e em scripts de grandes motores como Unreal e Blender.



Sintaxe Limpa

Parece inglês simples e direto. Sem símbolos complexos sem sentido.



Do Visual ao Texto: A Anatomia

⚡ +50 XP

SCRATCH (BLOCO)



Visual e limitado ao palco do Scratch. Excelente para aprender lógica inicial.

diga [Hello World!] por (2) segs

PYTHON (CÓDIGO PRO)



Texto executado direto no sistema operacional, servidores e sistemas reais.

```
print("Hello World!")
```



A Ferramenta do Desenvolvedor

⚡ +50 XP

- ⚡ No lugar da tela do Scratch, programadores usam **Editores de Código** (como VS Code ou PyCharm).
- ⚡ O código é escrito em arquivos com a extensão `.py`.
- ⚡ O **Terminal** é a janela preta onde seu programa imprime mensagens e roda de verdade!

 Editor de código neon futurista



O Mindset Hacker: Depuração

⚡ +50 XP



SyntaxError

Acontece quando você esquece de fechar aspas ou parênteses. O Python avisa na hora!



Traceback

É o relatório de erro do Python que diz exatamente em qual linha o bug aconteceu.



Resolução de Problemas

Errar a sintaxe no começo é normal. Programar é a arte de testar e corrigir!

Módulo 2: O Primeiro Olá Mundo no Terminal

Convertendo os blocos de fala em saídas de texto em Python!



A Função print()

⚡ +60 XP

SCRATCH

diga [Iniciando Sistema...] por (2) segs

Faz o balão de fala aparecer no ator no palco.

PYTHON

```
print("Iniciando Sistema ... ")
```

A função `print()` exibe qualquer mensagem no console/terminal.



O Segredo das Aspas (Strings)

⚡ +60 XP

- ⚡ Textos em programação são chamados de **Strings**.
- ⚡ Em Python, todo texto dentro do `print()` DEVE estar entre aspas duplas `" "` ou simples `' '`.
- ⚡ Se você esquecer as aspas, o Python achará que é uma variável e dará erro!

Maneira Correta:

```
print("Bem-vindo ao Python Pro!")
```

Erro de Sintaxe (Sem aspas):

```
# print(Bem-vindo) ← SyntaxError!
```

- ⚡ No Scratch, você precisa colocar vários blocos **diga** em sequência.
- ⚡ Em Python, cada `print()` escreve uma linha e pula para a próxima automaticamente!

```
print("=====")  
print(" CARREGANDO GAME ... ")  
print("=====")
```



Comentários: Anotações Invisíveis

⚡ +60 XP

- ⚡ Comentários servem para explicar o que seu código faz para outros desenvolvedores.
- ⚡ O computador ignora totalmente as linhas com comentário!
- ⚡ Em Python, usamos o símbolo de hashtag #.

```
# Este é um comentário de configuração  
velocidade = 100 # Define a velocidade inicial  
  
print("Jogo iniciado!")
```



Python Como Calculadora Instantânea

⚡ +60 XP

SCRATCH

(15) + (25)

Exige o bloco verde de operador inserido em um bloco diga.

PYTHON

```
print(15 + 25) # Imprime 40  
print(100 * 2) # Imprime 200
```



Juntando Texto e Números

⚡ +60 XP



No Scratch, usamos o bloco verde **junte [] com []**.



Em Python, você pode separar itens dentro do `print()` por vírgula!

```
print("Sua pontuação atual é:", 500)  
print("Nível:", 5, "| Vidas:", 3)
```

Módulo 3: Caixas de Memória: Variáveis & Tipos de Dados

Armazenando informações dinâmicas como um desenvolvedor profissional!



Criando Variáveis em Python

⚡ +70 XP

SCRATCH

Precisa clicar em "Criar Variável" no menu e depois usar o bloco laranja.

mude [pontos] para (100)

PYTHON

Basta digitar o nome da variável seguido pelo sinal de igual =!

```
pontos = 100
```



Os 4 Tipos Principais de Dados

⚡ +70 XP

int (Inteiro)

Números inteiros sem vírgula/ponto.

```
vidas = 3
```

float (Decimal)

Números com ponto decimal (altura, preço).

```
velocidade = 5.5
```

str (String)

Textos e caracteres entre aspas.

```
player = "Ninja"
```



Tipo Booleano: True ou False

- ⚡ Em Python, um **Booleano** armazena apenas dois valores possíveis: **True** (Verdadeiro) ou **False** (Falso).
- ⚡ Sempre com a primeira letra **Maiúscula**!
- ⚡ Ideal para verificar status de jogo (Game Over, escudo ativo, boss vivo).

```
game_over = False
```

```
tem_chave = True
```

```
print("Possui chave para abrir a porta?", tem_chave)
```



Modificando Variáveis (+= e -=)

⚡ +70 XP

SCRATCH

adicione (10) a [pontos]

PYTHON PRO

```
# Forma Pro Abreviada:  
pontos += 10 # Soma 10 aos pontos atuais  
vidas -= 1 # Subtrai 1 vida
```



F-Strings: Formatação Mágica

⚡ +70 XP

- ⚡ A forma mais moderna e elegante de colocar variáveis dentro de frases em Python!
- ⚡ Coloque a letra **f** antes das aspas e coloque as variáveis dentro de chaves **{ }**.

```
nome = "Alex"  
nivel = 42
```

F-String em ação:

```
print(f"O jogador {nome} alcançou o Nível {nivel}!")
```



Regras de Ouro de Nomes de Variáveis

⚡ +70 XP

❌ PROIBIDO

Usar espaços ou começar com números.

```
1lugar = "Ana"
```

✅ SNAKE_CASE

Use letras minúsculas separadas por underline.

```
pontos_do_jogador = 50
```

💡 SIGNIFICATIVO

Dê nomes claros do que a variável guarda.

```
tempo_restante = 30
```

Módulo 4: Entrada de Dados: Falando com o Usuário

Capturando o teclado e criando sistemas interativos!



A Função input()

⚡ +80 XP

SCRATCH

pergunte [Qual o seu nick?] e espere

Guarda a resposta no bloco especial

resposta

PYTHON

```
nick = input("Qual o seu nick? ")  
print(f"Bem-vindo à guilda, {nick}!")
```



O Perigo do input() Padrão

⚡ +80 XP

- ⚡ TUDO o que o usuário digita no `input()` entra como TEXTO (String)!
- ⚡ Se você pedir a idade e tentar somar, o Python vai juntar os textos em vez de somar matematicamente!

```
idade = input("Sua idade: ") # Se digitar 15, vira "15"
```

```
# Tentando somar 1 ano:
```

```
# print(idade + 1) ← TypeError!
```



Conversão de Tipos (Type Casting)

- ⚡ Para converter o texto do input em número inteiro, envelopamos o `input()` com `int()`!
- ⚡ Se precisar de números com vírgula (peso, altura), usamos `float()`.

```
# Convertendo para Número Inteiro:  
idade = int(input("Sua idade: "))  
proximo_ano = idade + 1  
  
print(f"Ano que vem você terá {proximo_ano} anos!")
```



Mini-Projeto: Gerador de Cartão Dev

⚡ +80 XP

```
# --- GERADOR DE BADGE DE JOGADOR ---  
nome = input("Nome do Player: ")  
classe = input("Classe (Mago/Guerreiro/Dev): ")  
nivel = int(input("Nível inicial: "))  
  
print("\n=====")  
print(f"PLAYER: {nome} | CLASSE: {classe}")  
print(f"POWER LEVEL: {nivel * 100} XP")  
print("=====")
```



Sanitização de Texto: `.strip()` e `.lower()`

⚡ +80 XP

- ⚡ Usuários costumam digitar com espaços extras ou maiúsculas sem querer.
- ⚡ `.strip()` remove espaços inúteis do início e do fim.
- ⚡ `.lower()` transforma todo o texto em minúsculas!

```
resposta = input("Deseja continuar? (s/n):  
").strip().lower()  
  
# Se o usuário digitar " Sim ", vira "sim"!
```



Desafio Ninja do Módulo 4

⚡ +80 XP

Missão Hacker: Crie um programa em Python que peça o nome do jogo favorito do usuário, o preço normal dele e calcule o preço com **20% de desconto** usando matemática!

Módulo 5: Tomada de Decisão: O Cérebro em Python

Aprenda a criar caminhos alternativos e inteligência no código!



O Comando if (Se)

⚡ +90 XP

SCRATCH

se < [pontos] > (50) > então

PYTHON

```
if pontos > 50:  
    print("Você passou de fase!")
```



A Regra Sagrada: Identação (Os 4 Espaços)

- ⚡ No Scratch, os blocos entram dentro do bloco C do `se`.
- ⚡ Em Python, não existem blocos visuais. O Python sabe o que está dentro do `if` pelo **RECUO DE CÓDIGO** (Identação / 4 Espaços ou Tab)!
- ⚡ Sem a identação correta, o Python dá **IndentationError**!

```
if hp ≤ 0:  
    # Linha recuada! Está DENTRO do if  
    print("GAME OVER!")  
# Linha sem recuo! Executa SEMPRE  
print("Obrigado por jogar.")
```



Operadores de Comparação

⚡ +90 XP

`==`

Igual a (Atenção: 2 sinais de igual! 1 sinal atribui, 2 sinais comparam!)

`!=`

Diferente de

`>=` e `<=`

Maior ou Igual / Menor ou Igual



if ... else (Se ... Senão)

⚡ +90 XP

SCRATCH

se ... então ... senão

PYTHON

```
if idade ≥ 18:  
    print("Acesso liberado ao torneio Pro!")  
else:  
    print("Acesso apenas à categoria Junior.")
```

Múltiplas Escolhas com elif

- ⚡ Quando você tem mais de 2 opções, usamos o **elif** (abreviação de else + if).
- ⚡ Você pode encadear quantos **elif** precisar antes do ``else`` final!

```
opcao = int(input("1-Start | 2-Loja | 3-Sair: "))

if opcao == 1:
    print("Iniciando partida ... ")
elif opcao == 2:
    print("Abrindo loja de skins ... ")
else:
    print("Saindo do jogo.")
```



Validador de Senha Hacker

⚡ +90 XP

```
senha_secret = "python123"
tentativa = input("Digite a senha do servidor: ")

if tentativa == senha_secret:
    print(" [SUCCESS] ACESSO ROOT CONCEDIDO!")
else:
    print(" [DENIED] SENHA INCORRETA. ALARME DISPARADO!")
```

Módulo 6: Operadores Lógicos & Condições Compostas

Combinando múltiplas checagens para criar regras complexas!



O Operador and (E)

⚡ +100 XP

SCRATCH

< > e < >

PYTHON

```
# As DUAS condições precisam ser True:  
if nivel ≥ 10 and tem_passaporte:  
    print("Pode entrar na área VIP!")
```



O Operador or (OU)

⚡ +100 XP

SCRATCH

< > ou < >

PYTHON

```
# BASTA UMA condição ser True:  
if moedas ≥ 100 or vip_card:  
    print("Item comprado com sucesso!")
```



O Operador not (NÃO)

⚡ +100 XP

- ⚡ Inverte o valor booleano: `True` vira `False` e vice-versa.
- ⚡ Perfeito para checar estados negativos ("Se NÃO estiver morto...").

```
game_over = False

if not game_over:
    print("O jogo continua rodando ... ")
```



Sistema de Rank em Jogos

⚡ +100 XP

```
score = int(input("Sua pontuação: "))

if score ≥ 1000:
    rank = "RADIANTE"
elif score ≥ 500 and score < 1000:
    rank = "OURO"
else:
    rank = "BRONZE"

print(f"Seu Rank Oficial: {rank}")
```



Checando Intervalos Elegantes

- ⚡ Em Python, você pode encadear comparações como na matemática!
- ⚡ Em vez de usar `x >= 10 and x <= 20`, você pode escrever direto `10 <= x <= 20`!

```
nivel = 15

# Sintaxe Pythonica Incrível:
if 10 <= nivel <= 20:
    print("Você está na faixa Intermediária!")
```



Desafio do Guardião da Dungeon

⚡ +100 XP

Missão Pro: Escreva um algoritmo em Python onde o jogador só entra no chefe final se tiver mais de **100 HP** `and` tiver o item **"Espada Mágica"** `or` se for da classe **"Paladino"**!

Módulo 7: Loops: Repetição de Alto Desempenho

Automatizando tarefas repetitivas com os laços `for` e `while`!



O Loop for e a Função range()

⚡ +120 XP

SCRATCH

repita (5) vezes

PYTHON

```
for i in range(5):  
    print(f"Executando tiro número {i + 1}")
```



Como Funciona o `range(início, fim, passo)`?

```
range(5)
```

Conta de 0 até 4 (5 números no total).

```
range(1, 10)
```

Conta de 1 até 9 (para antes do último número!).

```
range(0, 100, 10)
```

Conta de 0 a 90 pulando de 10 em 10!



Loop Infinito: while True

⚡ +120 XP

SCRATCH

sempre

PYTHON (GAME LOOP)

```
while True:  
    print("Rodando Game Loop principal ... ")
```



Quebrando Loops com break

⚡ +120 XP

- ⚡ O comando **break** força a saída imediata de qualquer loop!
- ⚡ Ideal para interromper o jogo quando o jogador morrer ou digitar "sair".

```
while True:
    comando = input("Digite 'sair' para parar: ")
    if comando == "sair":
        print("Encerrando aplicação ...")
        break
```



Pulando Etapas com continue

⚡ +120 XP

⚡ O comando `continue` pula o restante da rodada atual e vai direto para a próxima volta do loop.

```
for num in range(1, 6):  
    if num == 3:  
        continue # Pula o número 3!  
    print(f"Número: {num}")
```



Simulador de Contagem Regressiva

⚡ +120 XP

```
import time # Módulo de tempo nativo do Python
```

```
print("LANÇAMENTO DO FOGUETE EM:")
```

```
for i in range(5, 0, -1):
```

```
    print(f"{i} ... ")
```

```
    time.sleep(1) # Espere 1 segundo
```

```
print("🚀 DECOLAGEM CONFIRMADA!")
```

Módulo 8: Coleções Espaciais: Listas e Arrays

Armazenando centenas de itens em uma única estrutura de dados!



Criando Listas em Python

⚡ +140 XP

SCRATCH

Usa a categoria especial "Listas" e exige criar no menu visual.

PYTHON

```
# Lista criada entre colchetes [ ]:  
inventario = ["Espada", "Escudo", "Poção de HP"]  
print(inventario)
```



O Índice Zero: A Regra de Ouro

⚡ +140 XP

- ⚡ No Scratch, o primeiro item da lista é o 1.
- ⚡ Em Python (e quase todas linguagens reais), o primeiro item fica na Posição 0!
- ⚡ Índice 0 = Item 1 | Índice 1 = Item 2!

```
armas = ["Pistola", "Rifle", "Laser"]
```

```
print(armas[0]) # Imprime: Pistola
```

```
print(armas[2]) # Imprime: Laser
```



Adicionando Itens: .append()

⚡ +140 XP

- ⚡ O método `.append()` adiciona um novo item ao final da lista instantaneamente.
- ⚡ Perfeito para armazenar loot coletado pelo jogador durante a partida!

```
mochila = [] # Lista vazia

mochila.append("Chave Dourada")
mochila.append("Gema Vermelha")

print(mochila) # ['Chave Dourada', 'Gema Vermelha']
```



Removendo e Tamanho: len() e .pop()

⚡ +140 XP

```
len(lista)
```

Retorna quantos itens existem na lista.

```
lista.remove("item")
```

Remove pelo nome exato do item.

```
lista.pop(0)
```

Remove o item pelo número de índice.



Percorrendo Listas com Loop for

⚡ +140 XP

```
guilda = ["Mestre_Code", "Cyber_Ninja", "Shadow_Dev"]

print("MEMBROS CONECTADOS:")
for membro in guilda:
    print(f"● {membro} [ONLINE]")
```



Sorteio de Loot: módulo random

⚡ +140 XP

```
import random

loots = ["10 Moedas", "Poção Rara", "Lixo", "Espada Lendária"]
recompensa = random.choice(loots)

print(f"Você abriu o baú e ganhou: {recompensa}!")
```

Módulo 9: Funções: Criando Nossos Próprios Comandos

Organizando código modular, limpo e reutilizável!



O Que É Uma Função em Python?

⚡ +160 XP

SCRATCH

É o bloco rosa "Meus Blocos" (Criar um Bloco Customizado).

PYTHON

```
# Definindo a função com def:  
def chamar_hud():  
    print("≡ HUD ATIVO ≡")
```



Definir vs Chamar uma Função

- ⚡ Definir com **def** ensina o Python a fazer a receita.
- ⚡ Chamar a função pelo nome executa a receita na hora necessária!

1. Definição:

```
def atacar():  
    print("⚔ Golpe Crítico ativado!")
```

2. Chamada:

```
atacar()  
atacar()
```



Parâmetros: Entradas da Função

⚡ +160 XP

⚡ Parâmetros são variáveis que a função recebe para personalizar o resultado!

```
def subir_nivel(player, novo_level):  
    print(f"👉 {player} subiu para o Nível {novo_level}!")  
  
subir_nivel("Cyber_Ninja", 10)  
subir_nivel("Hacker_X", 15)
```



O Retorno de Dados: return

⚡ O comando **return** faz a função calcular algo e devolver a resposta de volta para onde foi chamada!

```
def calcular_dano(ataque, defesa):  
    return ataque - defesa  
  
dano_causado = calcular_dano(80, 30)  
print(f"Dano resultante: {dano_causado} HP")
```



Princípio DRY (Don't Repeat Yourself)

⚡ +160 XP



Código Limpo

Se você colou o mesmo bloco de código 3 vezes, ele deve virar uma função!



Manutenção Fácil

Se precisar mudar uma regra, muda apenas dentro da função e altera o jogo todo.



Modularidade

Torna seu projeto profissional e pronto para trabalhar em equipe.



Mini-Gerador de Batalha em Função

⚡ +160 XP

```
import random

def girar_dado_d20():
    return random.randint(1, 20)

resultado = girar_dado_d20()
print(f"🎲 Seu dado D20 tirou: {resultado}")
```

Módulo 10: Projeto Final: RPG Cyberpunk & REVISÃO GERAL

Construindo um jogo real e REAFIRMANDO tudo o que aprendemos!



Arquitetura do Jogo & Mapa de Revisão

⚡ +250 XP

1. Status & Inventário

Revisa Variáveis (`int`, `str`), Listas (`[]`) e entrada com `input()`.

2. Game Loop Infinito

Revisa `while True`, decisões com `if/elif/else` e `break`.

3. Combate e Funções

Revisa `import random`, funções com `def`, `return` e matemática.



Etapa 1: Início & Variáveis do Player

⚡ +250 XP

🔄 REAFIRMANDO O APRENDIZADO (MÓDULOS 2, 3 E 4):

- `player = input()`: Captura texto do teclado (Módulo 4).
- `hp = 100` e `moedas = 20`: Variáveis do tipo Inteiro/int (Módulo 3).
- `inventario = ["Laser"]`: Estrutura de Lista para guardar múltiplos itens (Módulo 8).
- `f"..."`: F-String formatando texto e variáveis com chaves (Módulo 3).

```
import random # Módulo de números aleatórios

player = input("Digite seu Cyber Nick: ").strip()
hp = 100
moedas = 20
inventario = ["Laser Inicial"]

print(f"🎮 BEM-VINDO À NEO-CITY, {player}!")
```



Etapa 2: O Game Loop Principal

⚡ +250 XP

🔄 REAFIRMANDO O APRENDIZADO (MÓDULOS 5, 7 E 8):

- `while hp > 0`: Loop condicional que mantém o jogo rodando enquanto tiver vida (Módulo 7).
- `len(inventario)`: Função nativa para contar quantos elementos há na lista (Módulo 8).
- `if acao == "3": break`: Condicional de igualdade `==` com quebra de loop via `break` (Módulo 5 e 7).

```
while hp > 0:
    print(f"\nHP: {hp} | Moedas: {moedas} | Itens: {len(inventario)}")
    acao = input("1- Batalhar | 2- Ver Mochila | 3- Sair: ").strip()

    if acao == "3":
        print("Saindo da simulação...")
        break
```



Etapa 3: Sistema de Batalha & Loot

⚡ +250 XP

🔄 REAFIRMANDO O APRENDIZADO (MÓDULOS 3, 5 E 8):

- `random.randint(10, 35)`: Sorteia números dentro de um intervalo dinâmico (Módulo 8).
- `hp -= dano` e `moedas += 15`: Operadores de atualização rápida para decrementar/incrementar (Módulo 3).
- `elif acao == "2":`: Estrutura de escolha múltipla encadeada (Módulo 5).

```
elif acao == "1":  
    dano = random.randint(10, 35)  
    hp -= dano  
    moedas += 15  
    print(f"⚡ Você derrotou o Drone e ganhou 15 moedas, mas tomou {dano} de dano!")  
elif acao == "2":  
    print(f"🎒 Itens na Mochila: {inventario}")
```



Etapa 4: Funções & Modularidade no RPG

⚡ +250 XP

🔄 REAFIRMANDO O APRENDIZADO (MÓDULO 9):

- `def gerar_evento_surpresa()`: Criação de função reutilizável evitando repetição DRY (Módulo 9).
- `random.choice(eventos)`: Sorteia um item diretamente da lista de eventos (Módulo 8).
- `return`: Devolve o resultado sorteado direto para o loop principal do jogo (Módulo 9).

```
def gerar_evento_surpresa():  
    eventos = ["Baú com Kit Médico (+20 HP)", "Emboscada Hacker!", "Loja Secreta Aberta"]  
    return random.choice(eventos)
```

```
# Chamando a função durante a exploração:  
print(f"🔍 Evento encontrado: {gerar_evento_surpresa()}")
```



PARABÉNS, PYTHON DEVELOPER!

Você concluiu com sucesso a ponte dos blocos do Scratch para o código Python de verdade!

Status: DEVELOPER PRO UNLOCKED | Continue Código em Nosso Site!

Acesse nosso portal para mais desafios, projetos com Pygame e certificados oficiais!

Image Sources



https://i.ytimg.com/vi/k0uDj80pil4/hq720.jpg?sqp=-oaymwEhCK4FEIIDSFrYq4qpAxMIARUAAAAAGAEIAADIQj0AgKJD&rs=AO4n4CLBg6w1SfgJRJZvV-aWky-2G2d8_Kw

Source: www.youtube.com



https://images.stockcake.com/public/6/1/1/611c2836-f1f6-4d15-a60c-aa8227e2902f_large/futuristic-code-display-stockcake.jpg

Source: stockcake.com